Serverless Backup and Recovery for Microservices using AWS

R Hemanth Kumar
Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aie22138@bl.students.amrita.edu

Vishnu Prasad B
Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aie22164@bl.students.amrita.edu

Himanshu Yadav
Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aie22167@bl.students.amrita.edu

Beena B M
Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bm_beena@blr.amrita.edu

***Abstract*—In our research, we have implemented an efficient backup and recovery solution for serverless microservices architectures that relies on services like AWS Lambda, AWS API Gateway, and AWS S3. The goal is to deliver a disaster recovery system that provides high availability while minimizing downtime in serverless environments. The proposed solution provides automated backup of AWS resources, such as S3 buckets and their contents, as well as mechanisms to ensure smooth recovery of microservices and configurations from stored backups. With the help of API Gateway and a simple-to-use dashboard built using HTML, CSS, and JavaScript, design takes advantage of the AWS ecosystem. Users can conveniently trigger resource discovery, backup, and recovery operations using this dashboard. The dashboard offers seamless user experience without high-tech technical expertise by emphasizing ease of use and simplicity. CloudWatch tracks important parameters in real time and keeps track of system performance, issues, and visibility of metrics. At the time of recovery, the recovery process dynamically restores original resources from backup, whereas the process of backup creates timestamped copies of resources to maintain the availability of historical data. Experiments' outputs confirm that the system is stable and efficient in minimizing downtime and ensuring constant availability of microservices in a scalable and cost-effective way.**

***Index Terms*—Serverless Architecture, Disaster Recovery, AWS Lambda, Automated Backup, Microservices,Amazon Web Services.**

I. Introduction

In this digital era of today scalable, modular and responsive microservice architectures make the foundation for the majority of currrent applications. Sectors such as e-commerce, healthcare, banking and entertainment have adopted microservices in large numbers to manage the constantly rising expectations of their customers. But migrating to microservices has many challenges involved particularly when it comes to reliability and disaster recovery. Accessing resources can be a major headache for serverless systems. Companies who rely on real-time user interactions are particularly vulnerable to a single point of failure which can lead to issues such as data corruption, accidental deletions or misconfigurations that affect essential services. This can result in downtime, data loss and ultimately financial setbacks. In serverless architectures the traditional disaster recovery methods that work well for monolithic systems won't work well. The distributed resources used in these systems, such as AWS Lambda, S3, and API Gateway, can scale on demand but lack the structured architecture that conventional systems have. For serverless applications where speed and reliability are crucial, relying on manual backup and restore processes isn't practical—they're slow and often fall victim to human error. Tracing and recovering resources becomes trickier due to the distributed nature of serverless systems.

That's where we come in with an automated backup and recovery solution tailored for serverless microservices architecture to tackle these challenges head-on. Our solution provide a scalable, reliable, and efficient disaster recovery process specifically crafted for serverless applications. By leveraging AWS native services and tools like Lambda, S3, API Gateway and CloudWatch, our system covers resource discovery, backup generation and recovery processes. With performance metrics monitored and potential issues identified and resolved swiftly, real-time monitoring enhances the overall reliability of the system. By minimizing the need for periodic manual intervention our solution significantly cuts down on downtime and reduces the risk of data loss, all while prioritizing automation and ease of use. Even non-technical stakeholders can easily control and oversee the system because of an intuitive dashboard constructed using HTML, CSS, and JavaScript.Timestamped backups guarantee the conservation and recoverability of previous versions of resources. This approach represents a significant advancement in the field of serverless disaster recovery, addressing the specific challenges posed by distributed systems. In addition to ensuring the continuous availability of microservices, it provides enterprises with a scalable and cost-effective solution to safeguard their

operations in the face of unforeseen failures.

II. Related Works

This section explains the previous work done on the cloud computing for recovery. [1]Q Wang team's research includes The various kinds of clouds, including public, private, community, and hybrid clouds, indicate their own characteristics and security grades in the paper: public clouds are available to each user via the Internet; private clouds provide better security measures for an enterprise. Strategies in data backups are basically divided into full, incremental, and differential backups, where the former is resource-intensive but comprehensive and the latter saves much time and storage space. It focuses on the DELTA compression algorithm, which reduces storage requirements of files by computing their differences. The scheme is very useful in version history maintenance and optimization of data transmission. Experimental results show that DELTA compression plays an effective role in maintaining cloud data storage even under attack to guarantee the stability of data. The second part describes the LANFree backup technology for cloud data backup by adopting information traffic statistics and data pattern matching to enhance backup efficiency. [2] It emphasizes serverless architecture scaling and implementation across different cloud service providers such as AWS, Azure, and Google Cloud Services. The authors have worked on articles published between 2018 and 2022 to ensure the relevance of the selected literature. One striking observation, emanating from this study, is that most serverless architectures are used on AWS, thus potentially missing out on any features that other CSPs might have to offer. Furthermore, most of the frameworks are specific and focus on certain aspects of synchronizations in serverless architectures but do not have any sweeping solution for interoperability amongst CSPs. The reason is that the results showed the existence of trade-offs of scalability across various serverless implementations, with AWS being the dominant player. This research offers an aspect-of-scenario impact on scaling factors and brings out implications of various serverless architecture implementations across CSPs.[3]Cloud computing offers on-demand provision of computing infrastructure and resources through the Internet, which facilitates cost-effectiveness and flexibility. Three core services of cloud computing include Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). Key benefits include scalability, enhanced security, and easy access. A comparison of Amazon AWS, Microsoft Azure, and Google Cloud gives insights in their performance using defined criteria. The comparison laid focus on the areas where further research needs to be done energy efficiency and dynamic resource scheduling optimization. This paper emphasizes the relevance of cloud services and lays the groundwork for future improvements in cloud technologies. [4] Previous work in intrusion recovery systems has focussed on database systems using logging-based recovery; such aspects have been described in detail in many text books. More recent work has focussed on recovery mechanisms for distributed systems, which include databases and web applications, directly relevant to microservice architecture. Finally, the "three R's" model of Rewind, Repair, Replay as a way of undoing actions performed by operators influenced parts of µVerum. While the Warp system rolls back databases to a pre-intrusion state and applies compensating operations, µVerum does something different: applying compensating operations without rolling back. In contrast, Bezoar logs file system operations and therefore might be impractical for microservices due to their reliance on virtual machines. In contrast, Shuttle and Rectify target monolithic web applications, while µVerum is targeted at microservices. Unlike µVerum, Aire logs operations to recover from malicious requests but uses selective re-execution. This general view therefore clearly underpins the novel contributions of µVerum and the advances in the mechanisms of intrusion recovery in the area of micro-services. [5] The paper focuses on the shift from monolithic to microservices architecture. It tells how this shift decreases coupling because of application division into a variety of independent services. Distributed transaction management is tackled in the research, focusing on two design patterns: two-phase commit versus Saga. Saga is preferred in this case because of its simplicity and flexibility. Building further on the literature, the study applies the Saga pattern in handling compensation processes of services involved in distributed transactions and introduces a back-up mechanism for state store and recoveries during an interruption. The authors describe Anser-Saga, a library that enhances data consistency among microservices through service orchestration, supporting transaction integrity and performance by providing backup and restart points. Experimental results show that, compared with the DTM solution, Anser library performs better in error compensation and has a better general performance, thus proving effective for real-world microservices scenarios. [6] The paper reviews microservices architecture by examining some critical aspects. It describes that microservices depend on highly available, replicated databases—although putting these solutions in place is complex and expensive—because this would reduce the need for further backups. It explores aligning microservices around business departments and value streams, leveraging existing business policies in managing data anomalies effectively to enhance autonomous functioning and loose coupling. It introduces the BAC theorem to state that, during the service design, a critical trade-off exists among backup, availability, and consistency. The three factors cannot be maximized simultaneously, and the balance among them affects the system coupling, which this paper also elaborates on. Finally, it warns against data loss through rollback to older backups, as keeping consistency during recovery across microservices is difficult. In conclusion, the survey gives insights into the nuanced considerations needed in implementing backup strategies that satisfy microservices' operational requirements, with complexity kept well under control. [7] This paper reviews data protection in cloud storage with respect to a very important methodology of using XOR operations with a

seed file for data storage and retrieval. It makes it possible to recover lost files given such computations between the XORed file and the seed file. It shows that data in the cloud is now in danger from hypervisors and multi-tenancy, and needs a very strong system of safeguarding their protection from various threats. It has ways to ensure the integrity and authentication of data in the survey, which are the methods while encrypting, with preservation of the privacy of users and access to data. It also proposed simplifying backup processes and storing data at multiple locations for the purpose of providing improved recovery options in disaster situations. It even mentions security frameworks like NIST and HIPAA and promotes measures to be followed in accordance with these standards so as to establish concrete cloud security for both consumers and providers.[8] The paper surveys the growing trend toward the adoption of serverless computing as a lean cloud architecture. Authors pinpoint what they understand to be key benefits of serverless platforms for developers to run code without concerns for infrastructural management, making them flexible, scalable, and cost-efficient. Of these, the use of serverless functions is stressed in scenarios such as event-driven applications and microservices. Then, it also explores challenges involved with vendor lock-in, security concerns, and limitations in performance for long-running tasks. It provides insights into the optimization of serverless deployment and, last but not least, further research on the overcoming of obstacles to its ability to realize its full potential in modern cloud environments.[9]This paper explores key principles and trends in load balancing and cloud computing with a focus on AWS technology. It highlights the effective load balancing approaches, giving significant importance to algorithm selection for optimal traffic management. The studies indicate that the better performance by AWS results from the integration of auto-scaling and load-balancing features. Cloud provider comparisons, such as Azure, indicate that there is a difference in optimization tactics tailored to client requirements. Security flaws in Amazon EC2 are also addressed with offered solutions. The paper s shows the importance of cloud automation in terms of cost reduction and server maintenance simplification. It also explores the concept of load-balanced web server architecture through Cloud Formation Templates, which paves a pathway for further study on AWS cloud improvement. [10]This study deals with energy efficiency in cloud computing, focusing on task scheduling, load balancing, and models of optimization to improve resource utilization, reduce execution time, and improve quality of service. It probes energy-saving solutions such as ant colony-based workload placement and hybrid policies, emphasizing SLA compliance and sustainability through green cloud practices. The report categorizes energy management solutions in data centers and identifies limitations in integrating energy optimization with strong security measures, especially in the case of edge-cloud systems. It stresses the demand for safe, energy-efficient solutions that respond to the requirements of modern computation while reducing environmental implications related to carbon emissions.[11]This paper investigates the use of AWS cloud architecture, highlighting its security features, efficiency and business benefits. It focuses on the powerful security features of AWS, such as encryption and multi-factor authentication that solves the problems caused by large amounts of data. Its scalability and cost-effectiveness make it a perfect service for SMEs by providing infrastructure as a service (IaaS) with high computing needs at a low cost. New studies prove that customers' attitudes have changed and AWS collaborates with open-source communities to fully improve the quality of experience. Free tier services also make AWS much more available for entrepreneurs it strengthens its position as a flexible and safe alternative for modern organizations.[12]This study reviews many studies on mental health, focusing on technology-driven solutions and tailored care. Digital interventions, such as internet-based programs and apps, have proven helpful in reducing anxiety, particularly among young people. FDA approval and responses from healthcare providers further serve to illustrate the potential of digital mental health interventions. Studies consider how COVID-19 impacts mental health, loneliness, and depression. Along with the media's impact on the mental health of teenagers, advanced detection approaches, like machine learning for stress monitoring, would be discussed. Complete care practices would also be accentuated through tailoring frameworks such as Integrated Precision Medicine (IPM) doctor-patient communication and sleep in well-being.[13]This paper examines cloud-based e-commerce systems, focusing on the significance and enhancements of using cloud computing for contemporary businesses. It underlines the scalability and cost-effectiveness of cloud applications, which are considered crucial for organizations to remain agile and responsive to market changes. The literature investigates architectural considerations such as data management, security and performance optimization. It addresses how emerging technologies like IoT, AI, and data analytics are transforming logistics in e-commerce. The integration of eKart with AWS as scalable, reliable, and secured that has led to better user experiences and consumer loyalty through effective services like Amazon EC2 and S3. [14]This paper assesses cloud migration choices, particularly the R-Lane approach known as "6vRs of cloud migration." It explains various strategies: Rehost (lift and shift), Retain (explore existing cloud initiatives), Retire (retire old applications), and Replace (implement new applications). The vote also addresses the modernization choices to move workloads from the on-premises environment to AWS, which encompasses activities such as assessment, data migration, and deployment. The report further underscores the importance of using open-source tools in application migration so that it could be compatible with the cloud systems. In a nutshell, it emphasizes proper preparation and the right tools to achieve effective cloud transition. [15] The paper "Advancing Microservices Architecture with Dockers" considers a role of Docker containers in the microservices architecture. It considers resource allocation optimization tactics such as

ADMD, security problems raised by tools such as DIVDS, performance comparisons between containers and regular VMs, and much more. Underlining Docker's ability to efficiently use resources via sharing OS kernels, future trends include using machine learning to improve container management and investigating multi-cloud and serverless deployments. Additionally, green computing activities are explored, with a focus on optimizing resource utilization for sustainability. The report offers a complete analysis of Docker's impact on microservices architecture and future developments.

III. System Architecture

A. API Gateway

API Gateway serves as the entry point for all client requests, managing RESTful endpoints for tasks such as resource discovery, backup, and recovery.It protects against malicious inputs and unauthorized access by ensuring requests are properly routed, authenticated, and validated before they reach the backend. API Gateway offers a basic security and scaling layer by serving as a bridge between frontend user interface and backend Lambda functions, ensuring that every client request is properly managed. by offering monitoring and throttling.

B. AWS Lambda

The most critical component of the system, AWS Lambda, manages activities such as resource discovery, backup, and recovery. Since they charge by use, serverless, stateless processes are economical and scale automatically according to demand. Recoveries and backups are serviced with ease due to Lambda's integration with Amazon S3 for activities such as bucket creation, object transfer, and restore. Error handling and logging features come with every Lambda function to make it more reliable and easier to debug. IAM controls implementation minimizes security breach risk while providing Lambda functions access to only what they require.

C. Amazon S3

Backups and resources are stored mainly on Amazon S3, which has very high durability and availability. Having a `-backup-timestamp` as a suffix on backup bucket names allows easy monitoring and maintenance of older data. S3 is the ideal choice for companies with growing storage needs because it accommodates scalability, meaning efficient management of vast amounts of information. S3 event notifications also automate the system recovery and backup by invoking Lambda functions for specific actions.

D. CloudWatch

As it monitors and logs system activity, CloudWatch tracks data including API Gateway requests, Lambda invocations, execution times, and errors. Configured dashboards give real-time system performance insights that help to identify and fix likely issues. Alerts for critical events, such those for 4XX and 5XX issues, enable quick system failure reaction, so minimizing downtime. By storing the logs generated by Lambda functions, CloudWatch guarantees operational transparency and helps debugging. With its thorough data, CloudWatch enables managers to change the system and guarantee optimal performance over time.

E. User Dashboard

HTML, CSS, and JavaScript help the user interface to give a basic way for people to interact with the system. Through this dashboard, users can start backups, restore data, and activate resource discovery, so enabling even non-technical stakeholders to access challenging activities. Real-time log and resource status display makes monitoring and controlling backups simple. The dashboard features extra components including progress indicators, problem alerts, and activity summaries to guarantee users are always informed. The responsive design make it compatible with several devices.

F. IAM Configuration

System security depends on Identity and Access Management(IAM), which limit resource access and guarantees that only authorised actions are taken. Policies and roles help to maintain the less priority concept by ensuring that every service or user has the minimal access required to perform their responsibilities. Lambda functions for example, help narrow down the systems attack surface to just S3 buckets and CloudWatch logs. Additionally, API Gateway employs IAM based authentication to confirm user identities.

G. Backend Development

The backend is built using AWS Lambda and Python, with a focus on three key tasks: identifying resources, creating backups and recovering data. Lambda functions make use of the AWS SDK to interact with S3 handling tasks like listing buckets, copying objects and restoring them. Each function comes with error handling protocol to catch and address any mistakes that might occur. API Gateway securely forward requests from the user interface to these functions enabling real time processing of user actions.

H. Frontend Development

The user dashboard was crafted with a strong emphasis on user-friendliness and simplicity, utilizing HTML, CSS and JavaScript. With this interface, users can effortlessly do resource discovery, create backups and restore their data. The REST APIs provided by the API Gateway facilitate seamless communication between the frontend and backend in real time making sure that tasks are completed on time. Features like loaders, error notifications and success messages significantly improve the user experience by giving clear feedback on every action. Plus the dashboard's responsive design guarantees it works smoothly across various devices catering to a diverse audience.

I. Security Configuration

The development stage was focused on security and implemented multiple layers of security, including implementing IAM roles and policies to avoid conflicting access by multiple users. Lambda functions were restricted to accessing only certain resources, such as CloudWatch logs, S3 data buckets, and allow lists. The API Gateway used IAM-based authentication to limit backend access to unauthenticated users. The Serverless Backup and Recovery System included Lambda functions implementing three steps: (1) discovering resources, (2) generating backups and (3) recovering resource state. CORS policies were used to prevent access to sensitive data and limit data communications with the frontend/backend for data protection.

J. Monitoring and Logging

CloudWatch offers system visibility by monitoring request counts for API Gateway, Lambda function execution times, and failures. It provides insights for the administrator to detect issues as well as to resolve them. CloudWatch stores logs for troubleshooting as well as performance tuning.

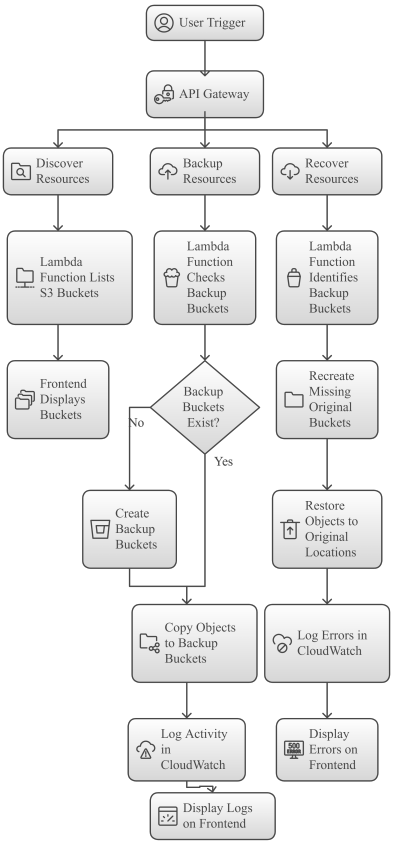


Fig. 1. How a Serverless Backup and Recovery System Works

Figure 1 shows serverless backup and recovery workflow, illustrating how Lambda functions route through the API Gateway for resource discovery, backup, and recovery.

IV. Algorithms

Algorithm 1: Discover_Assets

Input: Event containing S3 bucket information
Output: List of all buckets, backup buckets, and processed messages

```
Initialize all_buckets as an empty list;
Initialize backup_buckets as an empty list;
Initialize processed_messages as an empty list;
all_buckets ← List all S3 buckets using
 s3_client.list_buckets();
foreach bucket in all_buckets do
    if "-backup-" in bucket then
        Add bucket to backup_buckets;
foreach record in event.Records do
    message_body ← Extract record.body;
    Add {message_id: record.messageId, body:
     message_body} to processed_messages;
return {buckets: all_buckets, backups:
 backup_buckets, messages:
 processed_messages};
```

Algorithm 2: Backup_Assets

Input: List of all S3 buckets
Output: Backup completion message and backup details

```
Initialize backed_up_buckets as an empty list;
backup_suffix ← "-backup-" + current
 timestamp;
foreach bucket in all_buckets do
    if not bucket.endswith("-backup") then
        backup_bucket ← bucket +
         backup_suffix;
        Create backup_bucket using
         s3_client.create_bucket();
        pages ← Paginate objects in bucket using
         s3_client.get_paginator('lst_obj_v2');

        foreach page in pages do
            if page contains 'Contents' then
                foreach object in page.Contents
                 do
                    copy_source ← {Bucket:
                     bucket, Key: object.Key};
                    Copy object to backup_bucket
                     using
                     s3_client.copy_object();
        Add {source: bucket, backup:
         backup_bucket} to
         backed_up_buckets;
return {message: "Backup completed", backups:
 backed_up_buckets};
```

Algorithm 3: Recover_Assets

Input: List of backup buckets
Output: Recovery completion message and recovered bucket details

```
Initialize recovered_buckets as an empty list;
foreach backup_bucket in backup_buckets do
    original_bucket ← Extract name from backup_bucket by splitting on "-backup-";
    if original_bucket not in all_buckets then
        Create original_bucket using s3_client.create_bucket();
    pages ← Paginate objects in backup_bucket using s3_client.get_paginator('lst_obj_v2');

    foreach page in pages do
        if page contains 'Contents' then
            foreach object in page.Contents do
                copy_source ← {Bucket: backup_bucket, Key: object.Key};
                Copy object to original_bucket using s3_client.copy_object();
    Add {backup: backup_bucket, recovered_to: original_bucket} to recovered_buckets;
return {message: "Recovery completed", recovered: recovered_buckets};
```

V. Results

Our results demonstrate that implementing automated backups and swift recovery processes significantly minimizes downtime in serverless environments. This approach reduces operational overhead and enhances the resilience of cloud-native microservices. The solution is particularly well-suited for scalable, event-driven architectures where rapid recovery from failures is essential.

In Fig. 2, we illustrate the AWS Console setup on user interface, showcasing the configuration of resources and services used in the implementation. This includes the integration of AWS Lambda, DynamoDB, and S3, with proper configurations for automated backups and recovery workflows.

Fig.3 displays the IAM Manager interface in AWS, demonstrating the configuration of roles and policies that ensure secure and controlled access to resources for the backup and recovery processes.

Fig. 4 highlights the Lambda functions responsible for the automation of discovery, backup, and recovery tasks. It visually depicts the event-driven workflow and how each function interacts with AWS resources.

Fig. 5 shows the API response when the system successfully retrieves asset details (status code 200). The front-end's 'recoverAssets' function dynamically recovers the assets using the backup buckets while the 'logOutput' confirms the operation's success with a message indicating asset discovery was completed. 'discoverAssets' function dynamically appends the 'assetsList' with the retrieved information, while the 'logOutput' confirms the operation's success with a message indicating asset discovery was completed. 'backupAssets' function dynamically creates backup of all the assets with necessary prefix ('-backup-(time)) with the retrieved information, while the 'logOutput' confirms the operation's success with a message indicating asset discovery was completed.

Fig 6 demonstrates the monitoring and visualization of API calls in the AWS Console. It showcases logs and metrics that help analyze the performance and identify potential issues during the backup and recovery operations. Finally, Fig. 7,The dashboard of an S3 bucket, used for storing backup items. Assuming the buckets with normal name will be stored in developers account and the buckets with the backup prefix are stored in backup account. With all the functionalities of discover, backup and recovery.

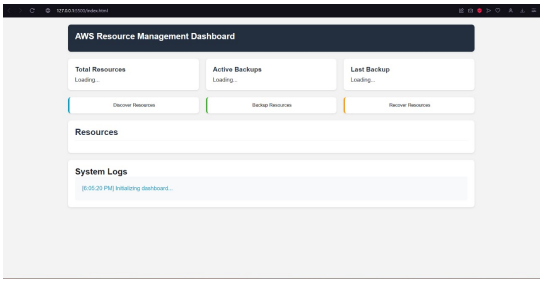


Fig. 2. User Interface

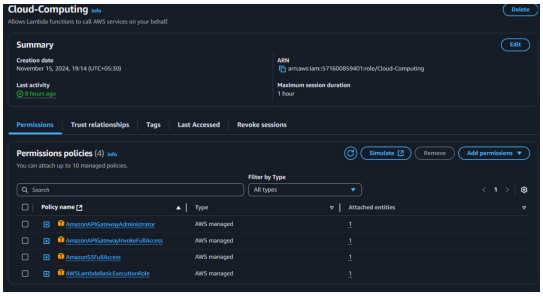


Fig. 3. IAM manager in AWS

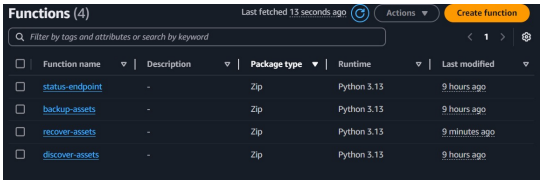


Fig. 4. Lambda Functions for Backup, Recover, Discover

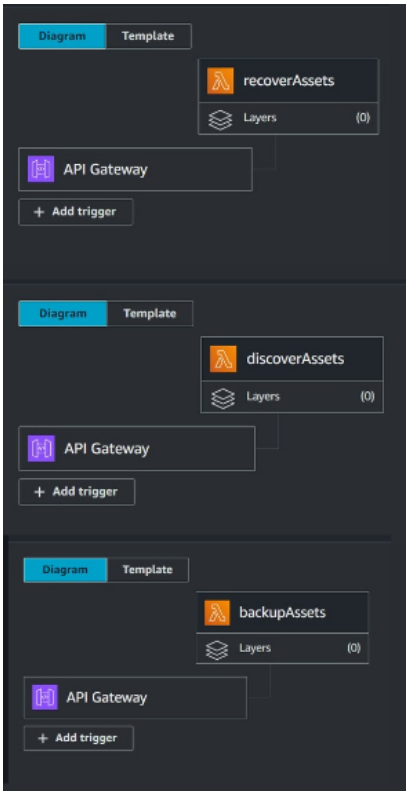


Fig. 5. API call Flow diagram

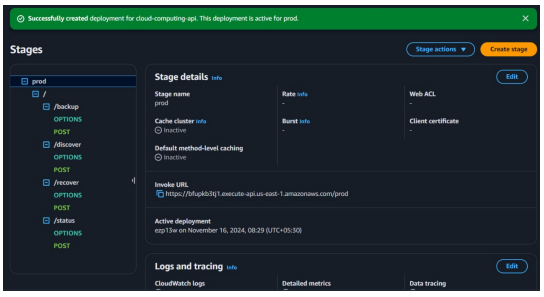


Fig. 6. API dashboard in AWS

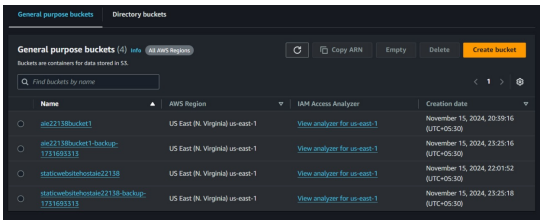


Fig. 7. S3 bucket in AWS for the Backup items

VI. Conclusion

In summary, the Serverless Backup and Recovery Solution has successfully demonstrated a highly scalable and responsive frame work which specifically addresses the disaster recovery requirements of a serverless microservices architecture. It utilizes AWS-native capabilities (Lambda, API Gateway, S3 and CloudWatch) to provide Vital services with continuous availability, sound performance and low downtime. Automation in resource discovery, back up and recovery means low manual effort, less operational burden and reduced risk of human error. This means the interface is intuitive with optimized processes so both technical and non-technical users can interact with it as users. Real-time monitoring and logging enhances system durability by eliminating potential issues. Timestamped backups support data integrity during historical recovery after an interruption has occurred. Not only can the solution address the companies' current microservices needs, it provides companies with a sustainable with cost-effective solution for consistent business continuity in the ever-changing tech industry. It paves the path for the next stages of disaster recovery planning by using serverless technologies from across the industries to develop plans that emphasize resilience, scalability and user experiences.

Our unique Serverless Backup and Recovery Solution helps to promote public safety by enabling services, like emergency response or healthcare, to remain functional through disaster events. It's also helpful to small businesses, non-profit organizations, and others by reducing waste, conserving energy and ensuring innovation. In this way, environmentally friendly initiatives also help support early stage companies, and society as a whole.

References

[1] Q. Wang, "Cloud Data Backup and Recovery Method Based on the DELTA Compression Algorithm," in *Proc. 2021 IEEE International Conference on Industrial Application of Artificial Intelligence (IAAI)*, Wuxi, China, 2021, pp. 1-6. doi: 10.1109/IAAI54625.2021.9699943.

[2] L. M. R. Cortes et al., "Serverless Architecture: Scalability, Implementations and Open Issues," in *Proc. 2022 International Conference on Computational Science and Computational Intelligence (CSCI)*, Bogotá, Colombia, 2022, pp. 1-6. doi: 10.1109/CSCI58124.2022.00239.

[3] P. Kaushik, S. Vashisht, A. M. Rao, S. Gupta, and D. P. Singh, "Cloud Computing and Comparison based on Service and Performance between Amazon AWS, Microsoft Azure, and Google Cloud," 2021 International Conference on Technological Advancements and Innovations (ICTAI), Greater Noida, India, 2021, pp. 268-273, doi: 10.1109/ICTAI53825.2021.9673425.

[4] D. R. Matos et al., "Verum: Intrusion Recovery for Microservice Applications," *IEEE Access*, vol. 11, pp. 1234-1245, July 2023, doi: 10.1109/ACCESS.2023.3298113.

[5] W.-T. Lee et al., "A High Availability Microservices Architecture Implementation using Saga and Backup Mechanism," in *Proc. 2023 10th International Conference on Dependable Systems and Their Applications (DSA)*, Kaohsiung, Taiwan, 2023, pp. 1-6. doi: 10.1109/DSA59317.2023.00063.

[6] G. Pardonet et al.,, "Consistent Disaster Recovery for Microservices: the BAC Theorem," *IEEE Cloud Computing*, vol. 5, no. 1, pp. 49-56, Jan./Feb. 2018, doi: 10.1109/MCC.2018.01185164.

[7] A. K. Das et al.,, "Data Protection for Backups and Recovery in Cloud Storage," in *Proc. 2022 International Conference on Smart Generation Computing, Communication and Networking (SMART GENCON)*, Karnataka, India, Dec. 23-25, 2022, pp. 1-6. doi: 10.1109/SMARTGENCON56628.2022.10084199.

[8] S. K. G. Dasari, "Serverless Cloud Functions - Opportunity in Chaos," 2022 International Conference on Computational Science and Computational Intelligence (CSCI), 2022, pp. 1330-1335. DOI: 10.1109/CSCI58124.2022.00239.

[9] R. Bundela, N. Dhanda, and R. Verma, "Load Balanced Web Server on AWS Cloud," 2022 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS), Lucknow, India, 2022, pp. 1-6, doi: 10.1109/ICCCIS56430.2022.10037657.

[10] P. P. Reddy, P.et al., "Improvising Energy Efficiency of Heterogeneous Servers Using AWS Services and Machine Learning Approaches," *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, Bengaluru, India, 2024, pp. 1-6, doi: 10.1109/ICCCNT61001.2024.10724212.

[11] M. K. Bali,et al., "Implementation of AWS Cloud Infrastructure," 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), Delhi, India, 2023, pp. 1-6, doi: 10.1109/ICCCNT56998.2023.10307993.

[12] D. Chennupalle, S.et al., "Enhancing Mental Health with AWS Cloud Services for Mental Well Hub," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 2024, pp. 1-6, doi: 10.1109/ICCCNT61001.2024.10724077.

[13] G. Ashrith Sai,et al.., "eKart: A Cloud-Based E-commerce Platform for Shopping deployed on AWS," 2024 International Conference on Computational Intelligence and Computing Applications (ICCICA), 2024, pp. 1-6, doi: 10.1109/ICCICA60014.2024.10585179.

[14] R. C. Pushpaleela, et al., "Application Modernization Strategies for AWS Cloud," 2022 1st International Conference on Computational Science and Technology (ICCST), Chennai, India, 2022, pp. 1-6, doi: 10.1109/ICCST55948.2022.10040356.

[15] K. P. Sah,et al., "Advancing of Microservices Architecture with Dockers," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bengaluru, India, 2024, pp. 1-6, doi: 10.1109/ICCCNT61001.2024.10724035.

[16] A. Rao, F. Ashik, I.et al., "Cloud-Integrated Blockchain Solution for Agricultural Sales Tracking," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 2024, pp. 1-6, doi: 10.1109/ICCCNT61001.2024.10726066

[17] A. Maharjan,et al., "AWS Server Less Luggage Tracking System Integrating IOT," 2024 2nd International Conference on Networking, Embedded and Wireless Systems (ICNEWS), Bangalore, India, 2024, pp. 1-8, doi: 10.1109/ICNEWS60873.2024.10730973.

[18] G. Venkat Sai Kumar,et al., "Revolutionizing Healthcare Waste Management: Leveraging Blockchain For Enhanced Efficiency and Transparency," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 2024, pp. 1-5, doi: 10.1109/ICCCNT61001.2024.10725300.

[19] P. Lertpongrujikorn and M. A. Salehi, "Tutorial: Object as a Service (OaaS) Serverless Cloud Computing Paradigm," 2024 IEEE 44th International Conference on Distributed Computing Systems Workshops (ICDCSW), Jersey City, NJ, USA, 2024, pp. 5-8, doi: 10.1109/ICDCSW63686.2024.00006.

[20] Y. Su, L. Sang,et al., "A Review of Multimedia Video Services Based on Serverless Cloud Computing," 2023 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), Beijing, China, 2023, pp. 1-5, doi: 10.1109/BMSB58369.2023.10211228.